Synthetic Drifting Streams over Binary Features: Regression and Classification

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

This paper specifies reproducible synthetic online regression and multiclass classification tasks for binary feature vectors. Concept drift shows up across real-world ML, yet most frameworks still assume stationary train/serve data; these tasks take nonstationarity as a first-class requirement with controllable drift axes rather than an afterthought. A synthetic setting makes it easy to experiment with controlled, reproducible scenarios—known latent mechanisms, seeded trajectories, and explicit scenario knobs—so adaptation can be studied without confounding from opaque real streams. The paper defines both synthetic environments, their independent sources of concept drift, the model boundary, and the prequential evaluation protocol for each task. It is a task and library specification, not a performance report.

1 Notation

ξ, d, m, σ Root seed, input width, number of components, and noise standard deviation.

x_t, g_t Current binary input and latent activity-gate states.

K, y_t Number of classes and observed class label (classification).

$s_{t,c}, \tilde{s}_{t,c}$ Noiseless and noisy class score (classification).

$y_t, \tilde{y}_t$ Noiseless and observed regression targets.

b_t, ϵ_t, σ Intercept, observation noise, and its standard deviation.

$f_i : \{0,1\}^d \to \{0,1\}, w_i$ Fixed binary indicator function and its fixed weight.

k_i, I_i, ν_i Clause degree, ordered selected-feature indices, and per-literal negation flags.

$k_{\min}, k_{\max}, p_\neg$ Clause degree bounds and negated-literal probability.

$q_{\max}, p_{\min}, p_{\max}$ Maximum parent count and CPT range.

$p_x, p^x_{\text{sample,min}}, p^x_{\text{sample,max}}$ Input distribution-sampling probability and node-sampling-probability range.

$p_g, p^g_{\text{sample,min}}, p^g_{\text{sample,max}}$ Gate distribution-sampling probability and node-sampling-probability range.

$w_{\min}, w_{\max}, b_{\min}, b_{\max}$ Weight and intercept ranges.

p_b Intercept-sampling probability.

S_t^x, S_t^g Input- and gate-distribution sampling indicators.

$S_{t,j}^x, S_{t,j}^g$ Input-node and gate-node sampling indicators.

S_t^b Intercept-sampling indicator.

2 Regression task

At time t, an environment emits a binary vector $x_t \in \{0,1\}^d$ and an observed scalar target $\tilde{y}_t$. The environment also maintains binary activity gates $g_t \in \{0,1\}^m$, one per target component, which are never supplied to a model. Keeping g_i hidden keeps activity latent: if gates were observed, $g_i f_i(x)$ would collapse to just another visible component function, and the same f_i could not turn on and off without becoming a distinct observed feature. A latent target is

$$y_t = b_t + \sum_{i=1}^{m} w_i g_{t,i} f_i(x_t), \qquad \epsilon_t \sim \mathcal{N}(0, \sigma^2), \qquad \tilde{y}_t = y_t + \epsilon_t,$$

where ϵ_t is zero-mean Gaussian noise. This form asks models to discover structure in input bits. Activity gates provide controllable levels of concept drift without rewriting the underlying components. Each $f_i : \{0,1\}^d \to \{0,1\}$ is a binary indicator (boolean component) defined as one conjunction of literals over a selected subset of input bits: each literal is either a coordinate or its negation, and all selected literals must be satisfied for $f_i(x) = 1$.

Fixed target construction. For each component, draw the clause degree

$$k_i \sim \text{DiscreteUniform}\{k_{\min}, \ldots, k_{\max}\},$$

sample distinct input indices

$$I_i \sim \text{Uniform}\left(\binom{\{1, \ldots, d\}}{k_i}\right),$$

and, independently for each selected index, draw a negation flag

$$\nu_{i,\ell} \sim \text{Bernoulli}(p_\neg) \qquad (\ell = 1, \ldots, k_i).$$

The component is

$$f_i(x) = \bigwedge_{\ell=1}^{k_i} \ell_{i,\ell}(x),$$

where $\ell_{i,\ell}(x) = x_{I_{i,\ell}}$ if $\nu_{i,\ell} = 0$ and $\ell_{i,\ell}(x) = \neg x_{I_{i,\ell}}$ otherwise. Weights are sampled once, for example $w_i \sim \text{Uniform}(w_{\min}, w_{\max})$, and remain fixed for the trajectory. The initial intercept is

$$b_0 \sim \text{Uniform}(b_{\min}, b_{\max}).$$

Thus functions and weights are fixed after initialization. Holding components fixed keeps drift attributable to gates, intercept, noise, and input-state dynamics, and lets evaluation ask how fast a model re-adapts to scenarios it has already faced—for example because it built and kept reusable representations rather than forgetting them. Output changes arise from activity-gate transitions and independent intercept sampling.

3 Input and gate state dynamics

Let $X_t = (X_{t,1}, \ldots, X_{t,d})$ be the input state and $G_t = (G_{t,1}, \ldots, G_{t,m})$ the activity-gate state. We describe input sampling below. Gate sampling uses the same procedure independently.

$$X_t \sim P_x.$$

The input DAG has d nodes. At initialization, sample its topological order:

$$\pi_x \sim \text{Uniform}(\mathcal{S}_d).$$

For each node, uniformly sample a parent subset from all subsets of earlier nodes in its topological order with at most $q_{\max}$ parents. Independently sample every conditional Bernoulli probability in its CPT uniformly from $[p_{\min}, p_{\max}]$. Sample X_0 ancestrally from this DAG, then sample its shared node-sampling probability:

$$p^x_{\text{sample}} \sim \text{Uniform}\left(p^x_{\text{sample,min}}, p^x_{\text{sample,max}}\right).$$

Apply this construction independently to G_t: use m nodes, $\pi_g \sim \text{Uniform}(\mathcal{S}_m)$, and the gate-specific p_g, $p^g_{\text{sample,min}}$, and $p^g_{\text{sample,max}}$. Neither topological order alters the model-visible input-coordinate order.

State updates. After an emitted sample, draw the input-distribution sampling indicator:

$$S^x_t \sim \text{Bernoulli}(p_x).$$

If $S^x_t = 1$, sample a new input DAG order, parents, CPTs, and p^x_{sample} while retaining X_t. Then sample every input node through

$$S^x_{t,j} \sim \text{Bernoulli}(p^x_{\text{sample}}) \quad (j \in \{1, \ldots, d\}),$$

then, in topological order, ancestrally sample nodes with indicator one under the current input DAG.

Each sampled node is drawn from its CPT conditional on its current parent values. This partial ancestral sampling intentionally does not guarantee that the current state follows the configured DAG distribution. The gate state uses this same update independently: draw $S^g_t \sim \text{Bernoulli}(p_g)$, retain G_t if it is zero or sample a new gate distribution if it is one, then sample gate nodes in order with shared probability p^g_{sample}. Each shared node-sampling probability controls persistence and is sampled when its distribution is initialized or sampled again.

4 Stream-generation procedure

Configuration

Run Root seed ξ determines all random draws; d and m set the input width and number of target components; σ sets the observation-noise standard deviation.

Function $k_{\min}$, $k_{\max}$, and $p_\neg$ determine the clause degree bounds and the probability that each selected literal is negated.

Distribution structure Parent limit $q_{\max}$ and CPT range $[p_{\min}, p_{\max}]$ bound each input and gate DAG's parents and conditional probabilities.

Input dynamics p_x controls new input-distribution sampling; $p^x_{\text{sample,min}}$ and $p^x_{\text{sample,max}}$ bound the shared input-node sampling probability.

Gate dynamics p_g controls new gate-distribution sampling; $p^g_{\text{sample,min}}$ and $p^g_{\text{sample,max}}$ bound the shared gate-node sampling probability.

Target scale $w_{\min}$, $w_{\max}$, $b_{\min}$, $b_{\max}$ bound weights and intercepts; p_b controls new intercept sampling.

Initialize: sample fixed functions f_i and weights w_i, initial intercept b_0, input and gate distributions, states X_0, G_0, and their shared node-sampling probabilities p^x_{sample} and p^g_{sample}.
For $t = 0, 1, \ldots$:

1. Set $o_t \leftarrow X_t$ and
$$\tilde{y}_t \leftarrow b_t + \sum_{i=1}^{m} w_i G_{t,i} f_i(X_t) + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, \sigma^2);$$
emit $(o_t, \tilde{y}_t)$.

2. Draw S^x_t and S^g_t. If either is one, resample the corresponding distribution; retain X_t or G_t.

3. For every input node j, draw $S^x_{t,j} \sim \text{Bernoulli}(p^x_{\text{sample}})$; in topological order, ancestrally sample nodes with indicator one to produce X_{t+1}. Repeat independently for every gate node using $S^g_{t,j} \sim \text{Bernoulli}(p^g_{\text{sample}})$ to produce G_{t+1}.

4. Draw $S^b_t \sim \text{Bernoulli}(p_b)$. If it is one, sample $b_{t+1} \sim \text{Uniform}(b_{\min}, b_{\max})$; otherwise set $b_{t+1} \leftarrow b_t$.

5 Drift mechanisms

Input and gate distribution sampling change their distributions abruptly. Neither sampling operation directly changes its family's current binary state, so latent activity continues from its pre-sampling value and is subsequently updated only by the usual partial ancestral sampling under the new distribution. Partial ancestral sampling provides persistent, within-family correlated states but does not necessarily follow either configured DAG distribution. Functions and weights do not drift. Activity-gate sampling and independent intercept sampling are the sources of output shifts. Gates remain unobserved activity variation. Oracle metadata is optional and is never supplied to a model.

6 Multiclass classification task

The classification variant reuses the observed binary input stream $x_t \in \{0,1\}^d$ and the same conjunction-of-literals function family $f_j : \{0,1\}^d \to \{0,1\}$ sampled once at initialization. For each class $c \in \{0, \ldots, K-1\}$, the environment maintains hidden activity gates $g_{t,c} \in \{0,1\}^m$, intercept $b_{t,c}$, and weights $w_{c,j}$ sampled independently at initialization and fixed thereafter. The noiseless class score is
$$s_{t,c} = b_{t,c} + \sum_{j=1}^{m} w_{c,j}\, g_{t,c,j}\, f_j(x_t).$$

Independent Gaussian noise perturbs each score, $\tilde{s}_{t,c} = s_{t,c} + \epsilon_{t,c}$ with $\epsilon_{t,c} \sim \mathcal{N}(0, \sigma^2)$, and the emitted label is

$$y_t = \operatorname*{argmax}_c \tilde{s}_{t,c},$$

with ties broken toward the smallest class index. Input drift matches the regression task. Gate distribution resampling, partial gate resampling, and intercept resampling are independent per class using the same probabilities p_g, $p^g_{\text{sample,min}}$, $p^g_{\text{sample,max}}$, p_b, $b_{\min}$, and $b_{\max}$ as in regression.

Stream generation. Initialize shared functions, input distribution, and per-class gate distributions, gate states, intercepts, and weight matrices. At each time step emit (x_t, y_t) from the current states and noisy scores, then apply the shared input update and independent per-class gate and intercept updates described above.

7 API and integration boundaries

Swappable generators, models, and metrics are required for fair prequential benchmarks; coupling them would bake evaluation assumptions into model code. The `binaml.environments` package owns generation, configuration, serialization, and state restoration through separate regression and classification modules. `binaml.models` owns online models and has no environment dependency. A model implements `predict(features)` and `update(target)`. The prediction call retains the features and any derived state; `update` applies the revealed target to that stored prediction. The `binaml.evaluation` package combines the two through predict-then-update prequential evaluation. Named benchmark scenarios only compose these independent components.

8 Reference online baselines

The library provides generic online baselines alongside the binary-feature models. For regression, the linear SGD baseline predicts with an affine function of the observed coordinates and the MLP baseline is a JAX one-hidden-layer ReLU network. For classification, the linear SGD baseline uses softmax cross-entropy and the MLP baseline uses the same JAX replay scaffold with a softmax head. JAX baselines are optional and are installed through the `benchmarks` dependency extra so ordinary library users do not need JAX.

All provided models use the same replay convention. After updating with a target, they retain the most recent B labeled samples and make S optimization updates on that window. Linear and MLP baselines both apply an averaged full-batch gradient update at each step. Reported comparisons must state each baseline's architecture, learning rate, regularization, replay size B, step count S, optimizer settings, and random seed.

9 Determinism and reproducibility

Every trajectory is determined by a validated JSON configuration and a single root seed ξ. The implementation splits that seed into stable PCG64DXSM substreams for initialization, sampling, drift, and noise. Snapshots include the full latent state and bit-generator states. Scenarios, generator versions, and configuration fingerprints are recorded with run artifacts.

10 Prequential evaluation protocol

For each emitted sample, an evaluator first records the model prediction, then exposes the target through `update`. Regression benchmarks compute mean squared error from the recorded pre-update predictions; classification benchmarks compute accuracy from the recorded pre-update class predictions. Benchmarks publish their horizon, scenario configuration, seeds, generator version, and separate total, prediction, and update timings; aggregate metrics alone are insufficient for replication.

Warm-up. A scenario may specify a non-negative `warmup_samples` count smaller than its horizon. Models predict and update these initial samples normally, but their errors and step timings are excluded from reported regression MSE and RMSE or classification accuracy, and from published timings. Time-series plots retain the full trajectory and mark the first evaluated sample with a vertical line so that the learning period remains visible.

11 Limitations and scope

These tasks are designed for controlled adaptation studies. They do not claim that synthetic drift matches a particular deployment, nor does this document make comparative performance claims. Results may be added only for versioned scenarios with repeatable baseline runs.